wallet-mobile Proje Rehberi

Klasörler, dosyalar ve şu anki kodun anlatımlı özeti

Proje: Expo (React Native) ile yazılmış harcama takip uygulaması · Expo SDK 57 · React 19 · TypeScript
Hazırlanma tarihi: 24 Ekim 2026 · Kapsam: `wallet-mobile` klasörü (backend ayrıca `wallet-be` içinde)

> **Bu belge kimin için?** Projeyi geliştiren ve kodun nasıl bir arada çalıştığını adım adım anlamak isteyen biri için. Her klasörün neden var olduğu, içindeki her dosyanın ne yaptığı ve dosyaların birbirini nasıl kullandığı anlatılıyor.

İçindekiler

1. Büyük resim: uygulama nasıl çalışıyor?
2. Klasör ağacı
3. Kök dizindeki dosyalar
4. `assets/` – görseller
5. `src/app/` – ekranlar ve gezinme (Expo Router)
6. `src/pages/` – sayfa bileşenleri
7. `src/components/` – tekrar kullanılan arayüz parçaları
8. `src/hooks/` – durum ve veri mantığı
9. `src/services/` – backend ile konuşma
10. `src/models/` – veri tipleri
11. `src/constants/` – ayarlar, tema ve etiketler
12. `src/utils/` – yardımcı fonksiyonlar
13. Uçtan uca akışlar: liste yükleme ve harcama kaydetme
14. Projede tekrar eden kalıplar ve kurallar
15. Bilinen sınırlamalar ve sıradaki adımlar

1. Büyük resim: uygulama nasıl çalışıyor?

wallet-mobile, harcamalarını kaydetmeni ve listelemeni sağlayan bir mobil uygulama. Aynı kod Android, iOS ve web tarayıcısında çalışıyor. Uygulama verileri kendisi saklamıyor. Tüm harcamalar Spring Boot ile yazılmış `wallet-be` backend'ine HTTP üzerinden gönderiliyor ve oradan PostgreSQL veritabanına yazılıyor.

Kod, her katmanın tek bir işi olacak şekilde katmanlara ayrılmış:

Katman (klasör)	Sorumluluğu	Örnek
`app/`	Hangi adreste hangi ekran açılacak, menü ve sekmeler	`(tabs)/expenses.tsx`
`pages/`	Bir ekranın görünümünü oluşturmak	`ExpenseListPage`
`components/`	Birçok ekranda kullanılan arayüz parçaları	`AppButton`, `ExpenseCard`
`hooks/`	Ekranın durumu (state) ve veri çekme mantığı	`useExpenses`
`services/`	Backend'e HTTP istekleri	`expenseService.getAll()`
`models/`	Verinin şekli (TypeScript tipleri)	`Expense`, `Currency`
`constants/`	Değişmeyen değerler: adresler, renkler, Türkçe etiketler	`API_BASE_URL`
`utils/`	Saf yardımcı fonksiyonlar (biçimlendirme, doğrulama)	`formatCurrency`

Bağımlılıklar **yukarıdan aşağıya** gider. Bir sayfa hook'u kullanır, hook servisi kullanır, servis de `apiClient`'ı kullanır. Ters yönde bağımlılık yoktur, örneğin servis hiçbir zaman bir sayfayı bilmez. Bu sayede bir katmanı değiştirdiğinde diğerleri etkilenmez.

```
Ekran (app/) → Sayfa (pages/) → Hook (hooks/) → Servis (services/) → apiClient → Backend
                     ↓                ↓                ↓
                components/       utils/          models/  constants/
```

2. Klasör ağacı

```
wallet-mobile/
├── app.json                 Expo uygulama ayarları
├── package.json             bağımlılıklar ve komutlar
├── package-lock.json        bağımlılıkların kilitlenmiş tam sürümleri
├── tsconfig.json            TypeScript ayarları, @/ kısayolu
├── eslint.config.js         kod kalitesi kuralları
├── expo-env.d.ts            Expo tip tanımları (otomatik)
├── AGENTS.md / README.md    proje notları
├── assets/                  ikonlar, splash görseli
├── node_modules/            indirilen paketler (otomatik)
├── .expo/                   Expo'nun geçici dosyaları (otomatik)
└── src/
    ├── app/                 ROTALAR (her dosya bir ekran)
    │   ├── _layout.tsx                  kök Stack
    │   ├── expenses/new.tsx             Yeni Harcama (modal)
    │   └── (drawer)/
    │       ├── _layout.tsx              soldan açılan menü
    │       └── (tabs)/
    │           ├── _layout.tsx          alt sekmeler
    │           ├── index.tsx            Ana Sayfa  ( / )
    │           └── expenses.tsx         Harcamalar ( /expenses )
    ├── pages/
    │   ├── home/            HomePage
    │   └── expenses/        ExpenseListPage, ExpenseFormPage
    ├── components/
    │   ├── common/          AppButton, AppInput, AppText, ChipSelect,
    │   │                    DateField, FormField, Screen
    │   └── expense/         ExpenseCard
    ├── hooks/               useTheme, useExpenses, useExpenseForm
    ├── services/            api/apiClient, expenseService, userService
    ├── models/              enums, expense, user
    ├── constants/           config, labels, theme
    └── utils/               format, expenseForm
```

> **index.ts dosyaları ne işe yarıyor?** Neredeyse her klasörde bir `index.ts` var ve içinde sadece `export * from './Dosya'` satırları bulunuyor. Buna *barrel* (toplama) dosyası denir. Böylece `import { AppButton } from '@/components/common/AppButton'` yerine kısaca `import { AppButton, AppText } from '@/components/common'` yazılabilir. Yeni bir dosya eklediğinde onu bu index'e de eklemelisin.

3. Kök dizindeki dosyalar

`package.json`

Projenin kimlik kartı. Kullanılan kütüphaneler ve sürümleri burada yazar. Öne çıkanlar:

- `"main": "expo-router/entry"`: Uygulama açılınca Expo Router devreye girer ve `src/app` klasörünü okuyarak ekranları kurar.
- `expo`, `react`, `react-native`: temel çatı.
- `expo-router`: dosya tabanlı gezinme. Sekmeler (Tabs) ve yan menü (Drawer) de bu pakette gelir.
- `react-native-gesture-handler`, `react-native-reanimated`, `react-native-worklets`: kaydırma hareketleri ve animasyonlar. Yan menü bunlarla çalışır.
- `@react-native-community/datetimepicker`: formdaki tarih seçici.
- `expo-symbols`: sekme ve menü ikonları.
- `expo-constants`: Expo'nun bağlandığı bilgisayar adresini okumak için kullanılıyor (bkz. `config.ts`).

`scripts` bölümündeki komutlar: `npm start` geliştirme sunucusunu başlatır, `npm run lint` kod kontrolü yapar, `npm run typecheck` tip kontrolü yapar.

> Yeni paket eklerken `npm install` yerine her zaman `npx expo install paket-adı` kullan. Bu komut Expo SDK 57 ile uyumlu sürümü seçer.

`package-lock.json`

Her paketin ve onların alt paketlerinin *tam olarak hangi sürümünün* kurulduğunu kaydeder. Elle düzenlenmez, npm otomatik günceller. Projeyi başka bir bilgisayarda kurduğunda aynı sürümlerin gelmesini sağlar.

`app.json`

Expo'nun uygulama ayarları: adı, ikonları, splash (açılış) ekranı, Android ve iOS'a özel ayarlar.

- `"userInterfaceStyle": "automatic"`: Telefon karanlık moddaysa uygulama da karanlık temaya geçer.
- `"scheme": "walletmobile"`: `walletmobile://...` biçimindeki derin bağlantılar için.
- `plugins`: Yerel (native) ayar gerektiren paketler buraya eklenir. Tarih seçici de burada.
- `experiments.typedRoutes`: `router.push('/expenses/new')` gibi adresleri TypeScript kontrol eder, yanlış yazarsan hata verir.
- `experiments.reactCompiler`: React Compiler açık. Gereksiz yeniden çizimleri otomatik azaltır.

`ios/` ve `android/` klasörleri projede yok. Expo bunları bu dosyadan otomatik üretir, elle oluşturulmamalı.

`tsconfig.json`

TypeScript ayarları. `"strict": true` en sıkı tip kontrolünü açar. `paths` bölümü `@/` kısayolunu tanımlar. `@/hooks` yazmak `src/hooks` demektir. Böylece `../../../hooks` gibi uzun göreli yollar yazmana gerek kalmaz.

`eslint.config.js`

Expo'nun hazır ESLint kurallarını kullanır. `npx expo lint` bu kurallara göre kodu tarar ve hook kurallarının ihlali, kullanılmayan değişken gibi hataları bulur.

`expo-env.d.ts`

Expo'nun tip tanımlarını projeye tanıtan otomatik dosya. Düzenlenmez ve git'e eklenmez.

`AGENTS.md, README.md, LICENSE`

`AGENTS.md` projede çalışan yapay zekâ asistanları için kuralları içerir: Expo dokümanını kontrol et, paketleri `expo install` ile ekle, işi bitirmeden lint ve typecheck çalıştır. `README.md` Expo şablonundan gelen tanıtım dosyası. `LICENSE` lisans metni.

`node_modules/, .expo/, .vscode/`

`node_modules/` npm'in indirdiği paketlerdir, silinse bile `npm install` ile geri gelir. `.expo/` Expo'nun önbelleği ve otomatik ürettiği tiplerdir. `.vscode/` editör ayarlarıdır. Hiçbiri elle düzenlenmez ve ilk ikisi git'e girmez.

4. assets/ – görseller

Uygulamanın ikonları ve açılış görselleri. `app.json` bu dosyalara yol vererek kullanır.

Dosya	Kullanıldığı yer
`images/icon.png`	Genel uygulama ikonu
`expo.icon/`	iOS ikonu (yeni Apple ikon biçimi)
`images/android-icon-*.png`	Android uyarlanabilir ikonunun ön, arka ve tek renkli katmanları
`images/splash-icon.png`	Açılışta mavi arka plan üzerinde görünen logo
`images/favicon.png`	Web sürümünde tarayıcı sekmesindeki ikon

5. src/app/ – ekranlar ve gezinme (Expo Router)

Bu klasör **özel** bir klasördür. Expo Router, buradaki her dosyayı otomatik olarak bir *rota*ya, yani ekrana dönüştürür. Dosyanın yolu ekranın adresi olur. Projedeki kural şu: `app/` içinde sadece rota dosyaları bulunur, ekranın asıl kodu `pages/` içindedir. Bu yüzden rota dosyaları tek satırdır.

Dosya adı kuralı	Anlamı
`index.tsx`	Bulunduğu klasörün kök adresi
`_layout.tsx`	Ekran değil. Aynı klasördeki ekranları saran gezinme yapısı (Stack, Tabs, Drawer)
`(parantezli)/`	Grup klasörü. Adrese yansımaz, sadece ekranları bir yapı altında toplar

İç içe gezinme yapısı

```
Stack  (app/_layout.tsx)
├── (drawer)            → Drawer  (sol menü)
│     └── (tabs)        → Tabs    (alt sekmeler)
│           ├── index      "Ana Sayfa"   adres: /
│           └── expenses   "Harcamalar"  adres: /expenses
└── expenses/new        → modal "Yeni Harcama"  adres: /expenses/new
```

En dışta bir **Stack** (yığın) var. Ekranlar üst üste açılır ve geri tuşu bir öncekine döner. Stack'in içinde yan menü (**Drawer**), menünün içinde de alt sekmeler (**Tabs**) bulunuyor. "Yeni Harcama" doğrudan Stack'e bağlı olduğu için hem menünün hem sekmelerin *üstünde* modal olarak açılır.

`app/_layout.tsx` kök layout

Uygulamanın en dış katmanı. Üç işi var:

1. `useColorScheme()` ile telefonun açık veya karanlık modda olduğunu öğrenir ve `ThemeProvider`'a uygun temayı verir. Başlık çubuğu ve arka plan renkleri buna göre ayarlanır.
2. `Stack` içinde iki ekran tanımlar. `(drawer)` için başlık gizlidir (`headerShown: false`), çünkü başlığı sekmeler kendisi çiziyor. `expenses/new` ise "Yeni Harcama" başlığıyla modal olarak açılır.
3. `StatusBar` ile saat ve pil çubuğunun rengini temaya göre otomatik ayarlar.

`app/(drawer)/_layout.tsx` sol menü

`expo-router/drawer`'dan gelen `Drawer` bileşeniyle soldan açılan menüyü kurar. `useTheme()` ile menünün arka plan rengini ve seçili veya seçili olmayan öğe renklerini temaya bağlar. Menünün kendi başlığı gizli, çünkü ☰ butonu sekmelerin başlığında duruyor. Şu an menüde tek öğe var: `(tabs)` grubu, "Ana Sayfa" etiketi ve ev ikonuyla. Menüye yeni öğe eklemek için bu klasöre bir ekran dosyası açıp burada bir `<Drawer.Screen>` yazılır.

`app/(drawer)/(tabs)/_layout.tsx` alt sekmeler

`Tabs` bileşeniyle ekranın altındaki sekme çubuğunu kurar.

- `headerLeft`: Her sekmenin başlığının soluna `DrawerToggleButton` (☰) koyar. Bu buton, bir üst katmandaki Drawer'ı açar.
- İki sekme tanımlı: **Ana Sayfa** (`index`) ve **Harcamalar** (`expenses`).
- İkonlar `expo-symbols`'ün `SymbolView` bileşeniyle çiziliyor. iOS'ta Apple SF Symbols (`house.fill`), Android ve web'de Google Material Symbols (`home`) kullanılıyor.
- Aktif sekme temanın ana rengiyle (mavi), diğerleri ikincil gri renkle görünür.

Hedeflenen 5 sekmeye ulaşmak için `(tabs)/` içine yeni dosya eklenir ve burada `<Tabs.Screen>` tanımlanır.

`app/(drawer)/(tabs)/index.tsx, expenses.tsx, app/expenses/new.tsx`

Üçü de tek satırlık rota dosyalarıdır. İlgili sayfa bileşenini `pages/`'den alıp varsayılan dışa aktarım (`export default`) olarak verirler:

```
import { HomePage } from '@/pages/home';
export default HomePage;
```

`index.tsx` → `HomePage` (uygulama açılınca ilk görünen ekran), `expenses.tsx` → `ExpenseListPage`, `expenses/new.tsx` → `ExpenseFormPage`.

6. src/pages/ – sayfa bileşenleri

Her ekranın görünümü burada yazılır. Sayfalar ekranı *çizer* ama veriyi kendileri çekmez veya saklamaz. O iş hook'lara bırakılır. Böylece sayfa kodu kısa ve okunur kalır.

`pages/home/HomePage.tsx`

Ana sayfa. Şimdilik bilerek boş. Sadece `<Screen />` döndürüyor, yani doğru arka plan rengine ve kenar boşluklarına sahip boş bir sayfa. İleride özet ve grafikler buraya gelecek.

`pages/expenses/ExpenseListPage.tsx`

Harcamalar sekmesi. Tüm veriyi tek satırla hook'tan alır:

```
const { expenses, loading, error, refresh } = useExpenses();
```

Sonra bu dört değere göre karar verir:

- **İlk yükleme** (`loading` true ve liste boş): ortada dönen `ActivityIndicator` gösterilir.
- **Hata varsa**: kırmızı hata mesajı ve "Tekrar dene" butonu gösterilir. Buton `refresh`'i çağırır.
- **Normal durum**: `FlatList` her harcama için bir `ExpenseCard` çizer. `FlatList` sadece ekranda görünen satırları çizdiği için uzun listelerde de hızlıdır. `onRefresh` sayesinde listeyi aşağı çekince yenilenir. Liste boşsa "Henüz harcama yok" yazar.

Altta her zaman "Harcama ekle" butonu durur. Bu buton `router.push('/expenses/new')` ile form modalını açar.

pages/expenses/ExpenseFormPage.tsx

Yeni harcama formu. Form durumunun tamamı `useExpenseForm` hook'unda tutulur. Sayfa sadece alanları çizer:

```
const { values, errors, submitError, submitting, setField, submit } =
  useExpenseForm(() => router.back());
```

Hook'a verilen `() => router.back()` fonksiyonu "kayıt başarılı olunca modalı kapat" demektir.

Alan	Bileşen	Not
Başlık	`AppInput`	En fazla 100 karakter
Tutar	`AppInput`	`decimal-pad` klavye, "450,50" yazılabilir
Para birimi	`ChipSelect`	TRY / USD / EUR / GBP, varsayılan TRY
Kategori	`ChipSelect`	17 kategori, Türkçe etiketlerle
Ödeme yöntemi	`ChipSelect`	6 yöntem
Tarih	`DateField`	Varsayılan bugün
Açıklama	`AppInput` (multiline)	İsteğe bağlı, en fazla 500 karakter

Seçenek listeleri (`currencyOptions` , `categoryOptions` , `paymentMethodOptions`) dosyanın üstünde, enum'lardan **bir kez** üretilir. Böylece enum'a yeni bir kategori eklenince form otomatik güncellenir. `KeyboardAvoidingView` klavye açıldığında alanların klavyenin altında kalmamasını sağlar. `ScrollView` form uzun olduğu için kaydırmayı sağlar. `keyboardShouldPersistTaps="handled"` ise klavye açıkken bir seçeneğe ilk dokunuşta tıklamanın çalışmasını sağlar. En altta varsa genel hata mesajı ve "Kaydet" butonu bulunur. Gönderim sırasında buton dönen simge gösterir ve kapalıdır.

pages/home/index.ts, pages/expenses/index.ts

Barrel dosyaları. Sayfaları dışa aktarır.

7. src/components/ – tekrar kullanılan arayüz parçaları

Birden fazla yerde kullanılan küçük arayüz bileşenleri. `common/` genel amaçlı parçaları, `expense/` harcamaya özel parçaları içerir. Hepsi renklerini `useTheme()` 'dan aldığı için karanlık modda otomatik uyum sağlar.

components/common/

`Screen.tsx`

Her sayfanın dış çerçevesi. Üç şey sağlar: temaya göre arka plan rengi, 16 piksellik iç boşluk ve `SafeAreaView` ile alttaki sistem çubuğunun (iPhone'daki çizgi gibi) altına içerik girmemesi. `children` isteğe bağlıdır, bu yüzden boş sayfa olarak da kullanılabilir.

`AppText.tsx`

Projedeki tüm yazılar React Native'in `Text` 'i yerine bununla yazılır. İki ek özelliği vardır:

- `variant` : `title` (28px kalın), `subtitle` (20px), `body` (16px, varsayılan), `caption` (14px). Yazı boyutları her yerde tutarlı olur.
- `color` : Temadaki bir rengin adı, örneğin `"danger"` veya `"textSecondary"` . Doğrudan renk kodu yazılmaz, böylece karanlık modda doğru renk otomatik seçilir.

`AppButton.tsx`

Ana buton. `title` , `onPress` , `loading` ve `disabled` alır. `loading` true ise yazının yerine dönen simge çıkar ve buton basılamaz. Bu sayede form iki kez gönderilemez. Basılıyken saydamlık 0.8, devre dışıyken 0.5 olur.

`FormField.tsx`

Her form alanının ortak iskeleti: üstte etiket, ortada içerik (`children`), altta varsa kırmızı hata mesajı. `optional` verilirse etiketin yanına "(isteğe bağlı)" eklenir. `AppInput` , `ChipSelect` ve `DateField` bunu kullanır. Böylece tüm alanlar aynı görünür.

`AppInput.tsx`

`FormField` + `TextInput` . React Native'in `TextInput` 'unun tüm özelliklerini alır (`...rest`) ve üstüne `label` , `error` , `optional` ekler. Hata varsa çerçeve kırmızı olur. `multiline` verilirse en az 96 piksel yüksekliğinde ve yazının üstten başladığı bir alan olur.

`ChipSelect.tsx`

Kısa bir listeden **tek seçim** yaptıran, yan yana dizilen ve sığmayınca alt satıra geçen "çip" butonlar. *Generic* bir bileşendir: `ChipSelect<T>` yazımı, hangi enum ile kullanılırsa `onChange`'in o tipte değer döndürdüğünü TypeScript'e söyler. Seçili çip mavi dolgulu görünür. `accessibilityRole="radio"` ekran okuyucular için eklenmiştir. Dışa aktarılan `ChipOption<T>` tipi `{ value, label }` çiftidir.

`DateField.tsx`

Tarih seçimi platforma göre farklı çalışır:

- **iOS**: `DateTimePicker` sayfanın içinde kompakt bir düğme olarak görünür, Türkçe (`tr-TR`).
- **Android ve web**: Seçili tarihi "24.10.2026" biçiminde gösteren bir alan görünür. Dokununca `DateTimePickerAndroid.open()` ile takvim penceresi açılır.

Kütüphanenin yeni API'si olan `onValueChange` kullanılıyor. Eski `onChange` artık önerilmiyor.

`common/index.ts`

Bu klasördeki tüm bileşenleri dışa aktarır.

components/expense/

`ExpenseCard.tsx`

Listedeki tek bir harcama satırı. Solda kalın başlık ve altında "Market · 24.10.2026" (kategori · tarih) bulunur. Sağda kırmızı renkle "-₺450,50" yazar. Kategori adı `CategoryLabels`'tan, tutar `formatCurrency`'den, tarih `formatDate`'ten gelir.

8. src/hooks/ – durum ve veri mantığı

Hook, adı `use` ile başlayan ve içinde React'in `useState`, `useEffect` gibi hook'larını kullanan fonksiyondur. Ekranın "beyni" burada durur: hangi veri var, yükleniyor mu, hata var mı, butona basınca ne olacak. Sayfalar sadece hook'un döndürdüğü değerleri çizer.

> **useState kısaca:** `const [deger, setDeger] = useState(baslangic)`. React değeri çizimler arasında saklar. `setDeger(yeni)` çağrılınca bileşen yeniden çizilir. Normal bir değişken bunu yapamaz, çünkü her çizimde sıfırlanır ve React değiştiğini fark etmez.

`useTheme.ts`

Telefon karanlık moddaysa `Colors.dark`'ı, değilse `Colors.light`'ı döndürür. Bileşenler renklerini her zaman buradan alır.

`useExpenses.ts`

Harcama listesini backend'den çeker. Döndürdüğü değerler: `expenses`, `loading`, `error`, `refresh`.

- **Üç state:** liste (başta boş), yükleniyor (başta true), hata (başta null).
- **`useFocusEffect`:** Ekran *her öne geldiğinde* çalışır: ilk açılış, modal kapandıktan sonra, başka sekmeden dönüş. `useEffect` olsaydı sadece ilk açılışta çalışırdı ve yeni eklenen harcama listede görünmezdi, çünkü sekmeler ekranı arka planda canlı tutar.
- **`ignore` bayrağı:** Ekrandan çıkılırsa veya yeni bir istek başlarsa, geç gelen eski cevap yok sayılır ve state'i bozmaz.
- **`refresh`:** `reloadKey`'i bir artırır. Bu değer effect'in bağımlılığı olduğu için istek yeniden atılır. Listeyi aşağı çekince ve "Tekrar dene"de kullanılır.

`useExpenseForm.ts`

Harcama formunun tüm durumunu ve gönderimini yönetir.

- `values`: Formun içeriği. Başlangıç değeri `createEmptyExpenseForm`'dan gelir.
- `errors`: Alan bazında hata mesajları, örneğin `{ amount: 'Tutar zorunlu' }`.
- `submitError`: Formun altında görünen genel hata. `submitting`: gönderim sürüyor mu.
- `setField(key, value)`: Bir alanı günceller ve o alanın hatasını siler. Kullanıcı düzeltince kırmızı mesaj hemen kaybolur. `setValues(current => ...)` biçimi her zaman en güncel değer üzerinden çalışmayı garanti eder.
- `submit()`: (1) `validateExpenseForm` ile kontrol eder, hata varsa durur ve istek gitmez. (2) `toExpenseRequest` ile veriyi backend biçimine çevirip `expenseService.create`'i çağırır. (3) Başarılıysa `onSaved`'ı çağırır ve modal kapanır. (4) `ApiError` gelirse backend'in alan hatalarını ilgili inputların altına, genel mesajı butonun üstüne yazar. (5) `finally` ile her durumda `submitting`'i false yapar.

`hooks/index.ts`

Üç hook'u dışa aktarır.

9. src/services/ – backend ile konuşma

Backend'e giden tüm HTTP istekleri bu klasörden geçer. Hook'lar ve sayfalar `fetch`'i hiçbir zaman doğrudan çağırmaz. Adres, header, zaman aşımı ve hata işleme tek bir yerde toplandığı için değişiklik gerektiğinde sadece burası güncellenir.

`api/apiClient.ts` tüm isteklerin geçtiği yer

`request(method, path, body)` fonksiyonu adım adım şunları yapar:

1. Bir `AbortController` oluşturur ve 15 saniyelik zamanlayıcı kurar. Süre dolarsa isteği iptal eder.
2. `fetch(API_BASE_URL + path)` ile isteği gönderir. Header'lar `Content-Type` ve `Accept: application/json`. Gövde `JSON.stringify` ile metne çevrilir.
3. Cevap başarısızsa (`!response.ok`, yani 4xx veya 5xx) `toApiError` backend'in hata gövdesini okur ve `ApiError` fırlatır.
4. 204 (içeriksiz cevap, örneğin silme sonrası) gelirse boş döner. Diğer durumlarda JSON'u çözüp döndürür.
5. Ağ hatalarını Türkçe mesajlara çevirir: zaman aşımında "Sunucu yanıt vermedi", bağlantı yoksa "Sunucuya bağlanılamadı". Bu durumlarda `status` 0 olur.
6. `finally` ile zamanlayıcıyı her durumda temizler.

`ApiError` sınıfı JavaScript'in `Error`'unu genişletir ve üç bilgi taşır: `message`, `status` (HTTP kodu) ve `fieldErrors` (alan bazında doğrulama mesajları). **`ProblemDetail`** tipi, backend'deki `GlobalExceptionHandler`'ın döndürdüğü `{ detail, errors }` biçimini tanımlar. Dosyanın sonundaki `apiClient` nesnesi `get`, `post`, `put` ve `delete` kısayollarını sunar.

`expenseService.ts`

Harcama uç noktalarının (endpoint) listesi. Her fonksiyon tek satırdır:

Fonksiyon	HTTP isteği	Kullanıldığı yer
`getAll()`	`GET /api/expenses`	useExpenses
`getById(id)`	`GET /api/expenses/{id}`	henüz kullanılmıyor
`create(expense)`	`POST /api/expenses`	useExpenseForm
`update(id, expense)`	`PUT /api/expenses/{id}`	henüz kullanılmıyor
`remove(id)`	`DELETE /api/expenses/{id}`	henüz kullanılmıyor

`apiClient.get<Expense[]>` gibi yazımlardaki `<...>`, dönen verinin tipini TypeScript'e bildirir.

`userService.ts`

Kullanıcı uç noktaları: `getById` ve `update`. Hazır duruyor ama henüz hiçbir ekran kullanmıyor. Profil ekranı yapılınca kullanılacak.

`services/index.ts`

`apiClient`, `expenseService` ve `userService`'i dışa aktarır.

10. src/models/ – veri tipleri

Uygulamada dolaşan verinin *şekli*. Bunlar sadece TypeScript tipleridir, çalışan kod üretmezler (enum'lar hariç). Yanlış alan adı kullanırsan `tsc` hata verir. Hepsi backend'deki Java sınıflarının birebir aynasıdır.

`enums.ts`

Backend'deki `com.fin.wallet.entity.enums` ile aynı değerler:

- `Currency`: TRY, USD, EUR, GBP
- `ExpenseCategory`: FOOD, GROCERIES, TRANSPORTATION, HOUSING, UTILITIES, HEALTH, EDUCATION, ENTERTAINMENT, SHOPPING, CLOTHING, TRAVEL, SUBSCRIPTIONS, PERSONAL_CARE, GIFTS, INSURANCE, TAXES, OTHER (17 adet)
- `PaymentMethod`: CASH, CREDIT_CARD, DEBIT_CARD, BANK_TRANSFER, MOBILE_PAYMENT, OTHER

Değerler İngilizce ve büyük harflidir, çünkü backend bunları böyle bekler. Ekranda görünen Türkçe karşılıkları `constants/labels.ts`'tedir.

`expense.ts`

`Expense`: Backend'den gelen bir harcama. Backend'deki `ExpenseDTO`'nun aynası: `id`, `title`, `description` (boş olabilir), `amount`, `currency`, `category`, `paymentMethod`, `expenseDate` ("2026-10-24" biçiminde metin), `userId`, `createdAt`, `updatedAt`.

`ExpenseRequest`: Kaydederken gönderilen gövde. `Omit<Expense, 'id' | 'createdAt' | 'updatedAt'>`, yani Expense'in bu üç alan çıkarılmış hali. Bu alanları sunucu belirlediği için gönderilmezler.

`user.ts`

`User`: Backend'deki User entity'sinin aynası. Şifre alanı bilerek yok, çünkü şifre istemciye hiç gönderilmez.

11. src/constants/ – ayarlar, tema ve etiketler

`config.ts`

- **`API_BASE_URL`**: Backend'in adresi. `.env` dosyasında `EXPO_PUBLIC_API_URL` tanımlıysa o kullanılır. Tanımlı değilse adres `Constants.expoConfig.hostUri`'den (örneğin `192.168.1.46:8081`) alınır ve `http://192.168.1.46:8080/api` oluşturulur. Telefonda `localhost` telefonun kendisini gösterdiği için bu çözüm gerekti. Telefon Expo'ya hangi IP ile bağlandıysa backend'e de aynı IP ile gider.
- **`REQUEST_TIMEOUT_MS`**: 15000 ms. apiClient'ın zaman aşımı süresi.
- **`CURRENT_USER_ID`**: Geçici olarak 1. Login olmadığı için her harcama bu kullanıcıya yazılıyor. Bu kullanıcının veritabanındaki `users` tablosunda bulunması gerekir.

`theme.ts`

Tasarımın tek kaynağı. Bileşenlerde sayı veya renk kodu yazmak yerine buradaki adlar kullanılır:

- `Colors.light` / `Colors.dark`: primary (mavi), text, textSecondary, background, surface (kart zemini), border, danger (kırmızı), onPrimary (mavi üstündeki yazı rengi).
- `Spacing`: xs 4, sm 8, md 16, lg 24, xl 32 (piksel boşluklar).
- `Radius`: sm 6, md 12, lg 20 (köşe yuvarlaklıkları).
- `FontSize`: sm 14, md 16, lg 20, xl 28.

`as const` değerleri salt okunur yapar. `ThemeColors` tipi, `AppText`'in `color` özelliğinin sadece geçerli renk adlarını kabul etmesini sağlar.

`labels.ts`

Enum değerlerinin Türkçe karşılıkları: `CategoryLabels` (FOOD → "Yemek", GROCERIES → "Market"...) ve `PaymentMethodLabels` (CASH → "Nakit"...). `Record<ExpenseCategory, string>` tipi, enum'a yeni bir değer eklenip burada karşılığı yazılmazsa TypeScript'in hata vermesini sağlar. Böylece hiçbir değer çevirisiz kalmaz.

12. src/utils/ – yardımcı fonksiyonlar

React'ten bağımsız, girdi alıp çıktı veren *saf* fonksiyonlar. Ekran veya state bilmezler, bu yüzden test etmesi ve tekrar kullanması kolaydır.

format.ts

- `formatCurrency(450.5, 'TRY')` → "₺450,50". Tarayıcının ve telefonun `Intl.NumberFormat`'ını Türkçe ayarla kullanır.
- `formatDate('2026-10-24')` → "24.10.2026".
- `toIsoDate(date)` → "2026-10-24". Backend'in `LocalDate` için beklediği biçim. Yerel saat kullanıldığı için gece yarısına yakın saatlerde tarih bir gün kaymaz. `toISOString()` UTC'ye çevirdiği için bu kaymaya yol açabilirdi.

expenseForm.ts

- `ExpenseFormValues`: Formun yazım sırasındaki hali. Dikkat: `amount` burada *metindir* ("450,50"), sayıya ancak gönderirken çevrilir. Kategori ve ödeme yöntemi başta `null`'dır, yani seçilmemiş.
- `ExpenseFormErrors`: Her alan için isteğe bağlı bir hata mesajı.
- `createEmptyExpenseForm()`: Boş form. Para birimi TRY, tarih bugün.
- `parseAmount("450,50")` → 450.5. Virgülü noktaya çevirir, en fazla 13 tam ve 2 ondalık basamağa ve en az 0.01'e izin verir. Geçersizse `null` döner.
- `validateExpenseForm(values)`: Backend'deki `ExpenseDTO` kurallarının aynısını uygular ve Türkçe hata mesajları döndürür. Kurallar aynı olduğu için hatalar istek gitmeden telefonda yakalanır.
- `toExpenseRequest(values, userId)`: Form verisini backend'in beklediği JSON'a çevirir: boşlukları kırpar, tutarı sayıya, tarihi metne çevirir, boş açıklamayı `null` yapar. Sondaki `!` işaretleri "doğrulamadan geçtiği için bu değer boş olamaz" anlamına gelir.

13. Uçtan uca akışlar

A) Harcamalar sekmesi açılınca liste nasıl geliyor?

```
1. Kullanıcı "Harcamalar" sekmesine dokunur
2. ExpenseListPage çizilir → useExpenses() çağrılır
   loading = true → ekranda dönen simge
3. useFocusEffect çalışır → expenseService.getAll()
4. apiClient.get('/expenses')
   → fetch GET http://192.168.1.46:8080/api/expenses
──────────────────── ağ ────────────────────
5. Backend: ExpenseController.getAllExpenses
   → ExpenseQueryService → veritabanı (en yeni 20 kayıt)
   ← 200 + JSON dizisi
──────────────────── ağ ────────────────────
6. setExpenses(data), setLoading(false)
7. React sayfayı yeniden çizer → FlatList + ExpenseCard'lar
```

B) Formda "Kaydet"e basılınca ne oluyor?

```
1. AppButton onPress → submit()                (useExpenseForm)
2. validateExpenseForm(values)
   hata varsa → errors state'e yazılır, kırmızı mesajlar, İSTEK GİTMEZ
3. submitting = true → buton dönen simge, basılamaz
4. toExpenseRequest() → { title, amount: 450.5, expenseDate: "2026-10-24", ... userId: 1 }
5. expenseService.create() → apiClient.post('/expenses', body)
──────────────────── ağ ────────────────────
6. Backend: @Valid kontrol → ExpenseServiceImpl.save
   → kullanıcı id=1 bulunur → INSERT → 201 Created + kaydedilen harcama
──────────────────── ağ ────────────────────
7a. Başarılı: onSaved → router.back() → modal kapanır
    Liste ekranı öne gelir → useFocusEffect → liste yeniden çekilir
7b. Hata: ApiError → alan hataları inputların altına, genel mesaj butonun üstüne
8. finally → submitting = false
```

14. Projede tekrar eden kalıplar ve kurallar

- **Rota ile sayfa ayrımı:** `app/` içinde sadece tek satırlık rota dosyaları, asıl kod `pages/`'de.
- **Sayfa çizer, hook düşünür:** State ve veri çekme hook'larda. Sayfalar sadece görünüm.
- **Tek HTTP kapısı:** Her istek `apiClient`'tan geçer. `fetch` başka yerde çağrılmaz.
- **Sabit değer yazma, temadan al:** Renk, boşluk ve yazı boyutu her zaman `theme.ts`'ten gelir.
- **Backend ile ayna tipler:** `models/` backend DTO'larıyla birebir aynı tutulur. Backend değişince burası da güncellenir.
- **Çift doğrulama:** Kurallar hem telefonda (hızlı geri bildirim için) hem backend'de (güvenlik için) uygulanır.
- **Barrel import:** `@/components/common`, `@/hooks` gibi klasör adından içe aktarılır.

- **Türkçe arayüz, İngilizce kod:** Ekranda görünen metinler Türkçe, değişken ve fonksiyon adları ile yorumlar İngilizce.
- **İşi bitirmeden kontrol:** `npx tsc --noEmit` ve `npx expo lint` temiz geçmeli.

15. Bilinen sınırlamalar ve sıradaki adımlar

Konu	Şu anki durum	Olası çözüm
Sayfalama	Backend varsayılan olarak 20 kayıt döner, mobil sonraki sayfaları istemiyor. 20'den fazla harcamada eskiler görünmez.	`FlatList.onEndReached` ile sonsuz kaydırma, `X-Total-Count` header'ı ile bitişi anlama.
Kullanıcı	`CURRENT_USER_ID = 1` sabit.	Login ekranı ve token tabanlı kimlik doğrulama.
Ana sayfa	Boş.	Aylık toplam, kategori dağılımı, son harcamalar.
Sekmeler ve menü	2 sekme, menüde 1 öğe.	5 sekmeye tamamlama, menüye ayarlar/profil gibi öğeler.
Düzenle / sil	Servis fonksiyonları hazır ama arayüz yok.	Kartta dokununca düzenleme formu, kaydırınca silme.
Backend mesajları	Doğrulama mesajları İngilizce geliyor ("Validation failed").	Backend'de Türkçe mesaj dosyası (`messages_tr.properties`).
Tunnel modu	`expo start --tunnel` ile otomatik API adresi çalışmaz.	`.env` içinde `EXPO_PUBLIC_API_URL` tanımlamak.